CS 663: Digital Image Processing

Assignment 1 Report (Part 1.b, 1.c, 1.d)

Pushpendra Uikey

August 30, 2026

Contents

1 Image Subsampling and Interpolation

1.1 1(b) Image Enlargement using Nearest-Neighbor Interpolation

Implementation

The objective is to enlarge `random.png` from its original $M \times N$ dimensions to $300(M-1)+1 \times 300(N-1)+1$. To guarantee that the extreme boundary pixels of the original image map flawlessly to the extreme boundaries of the enlarged image, the inverse mapping algorithm calculates the continuous origin coordinate by scaling the current index by the ratio of the maximum valid indices $\frac{m-1}{M-1}$.

For Nearest-Neighbor interpolation, this continuous coordinate is rounded to the closest valid integer grid point using the `std::round` function. The pixel intensity at that discrete integer coordinate is then directly assigned to the new destination pixel. As required, the outputs are displayed using the "jet" colormap with a colorbar.

Observations

Because Nearest-Neighbor interpolation performs no structural smoothing or data blending, the enlarged output manifests as a series of massive, uniform color blocks. The transitions between distinct pixel intensities are perfectly sharp but heavily aliased, resulting in a pronounced "staircase" or checkerboard artifact across the spatial grid.

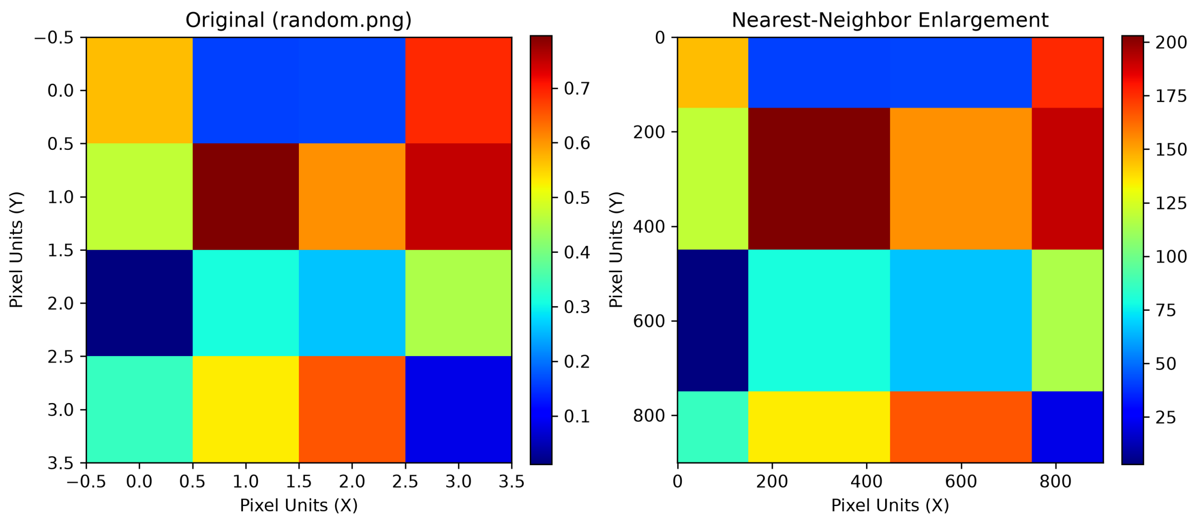

Figure 1: Nearest-Neighbor interpolation. The spatial mapping creates blocky, sharply aliased grid transitions.

1.2 1(c) Image Enlargement using Bilinear Interpolation

Implementation

Bilinear interpolation improves upon the nearest-neighbor approach by mathematically estimating the missing intensities between discrete grid points. The algorithm locates the 2×2 bounding box of original integer pixels immediately surrounding the inverse-mapped continuous coordinate. It computes the horizontal and vertical fractional distances (dx

and dy) from the top-left anchor pixel. These fractional distances are used as linear weights to interpolate the top and bottom rows horizontally, followed by a final vertical interpolation.

Observations

The bilinear output replaces the harsh blocks of the nearest-neighbor method with continuous, linear gradients. The transitions between the original data points are smoothly blended. However, because the interpolation is strictly linear, the resulting gradients exhibit "starburst" patterns and visible creases where the linear slopes abruptly change direction at the original pixel boundaries.

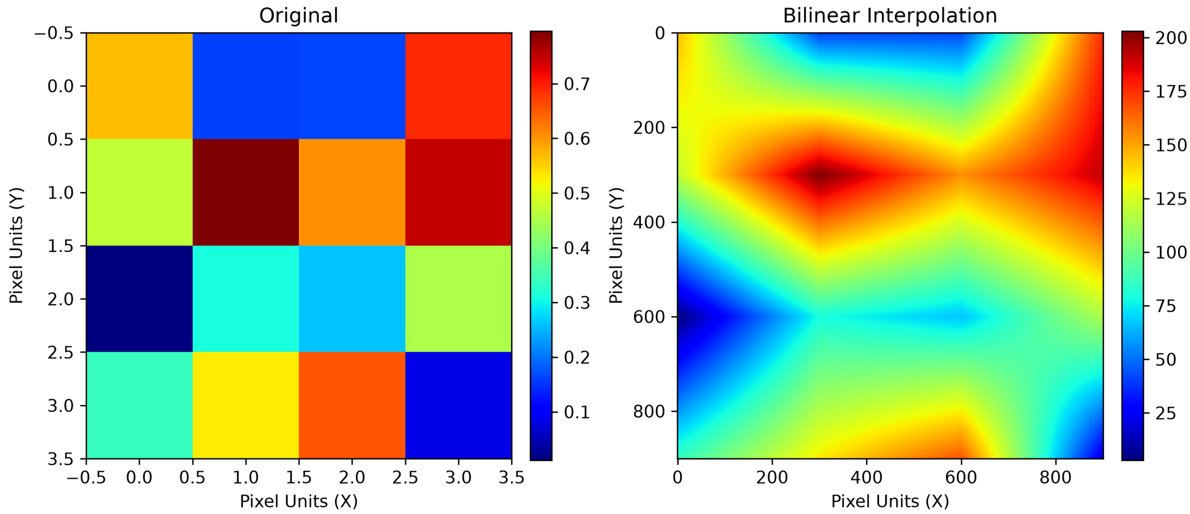

Figure 2: Bilinear interpolation. The harsh blocks are replaced by linear gradients, though sharp angular transitions (creases) remain visible.

1.3 1(d) Image Enlargement using Bicubic Interpolation

Implementation

Bicubic interpolation offers the highest fidelity by evaluating a 4×4 neighborhood containing 16 original pixels. The algorithm utilizes a specific piecewise cubic polynomial function to calculate the scalar weight of each surrounding pixel based on its absolute 1D distance from the target coordinate. The final intensity is derived by accumulating the sum of all 16 neighboring pixels, each multiplied by its computed 2D separable cubic weight. Boundary clamping is strictly enforced to safely process the overhanging 4×4 convolution windows at the image edges.

Observations

The bicubic interpolation yields a dramatically superior visual reconstruction. By fitting a continuous third-degree polynomial surface over the data, the algorithm completely eliminates the creases and angular artifacts present in the bilinear method. The resulting gradients are mathematically smooth, organic, and visually seamless, representing the most accurate spatial scaling of the original random data grid.

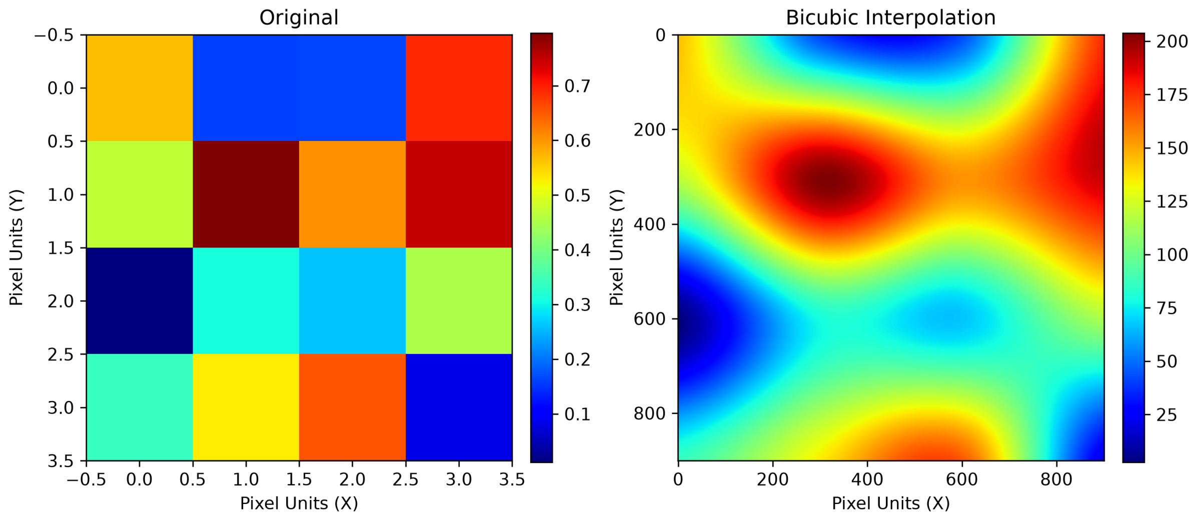

Figure 3: Bicubic interpolation. The higher-order cubic convolution creates a perfectly smooth, organic gradient surface across the entire matrix.